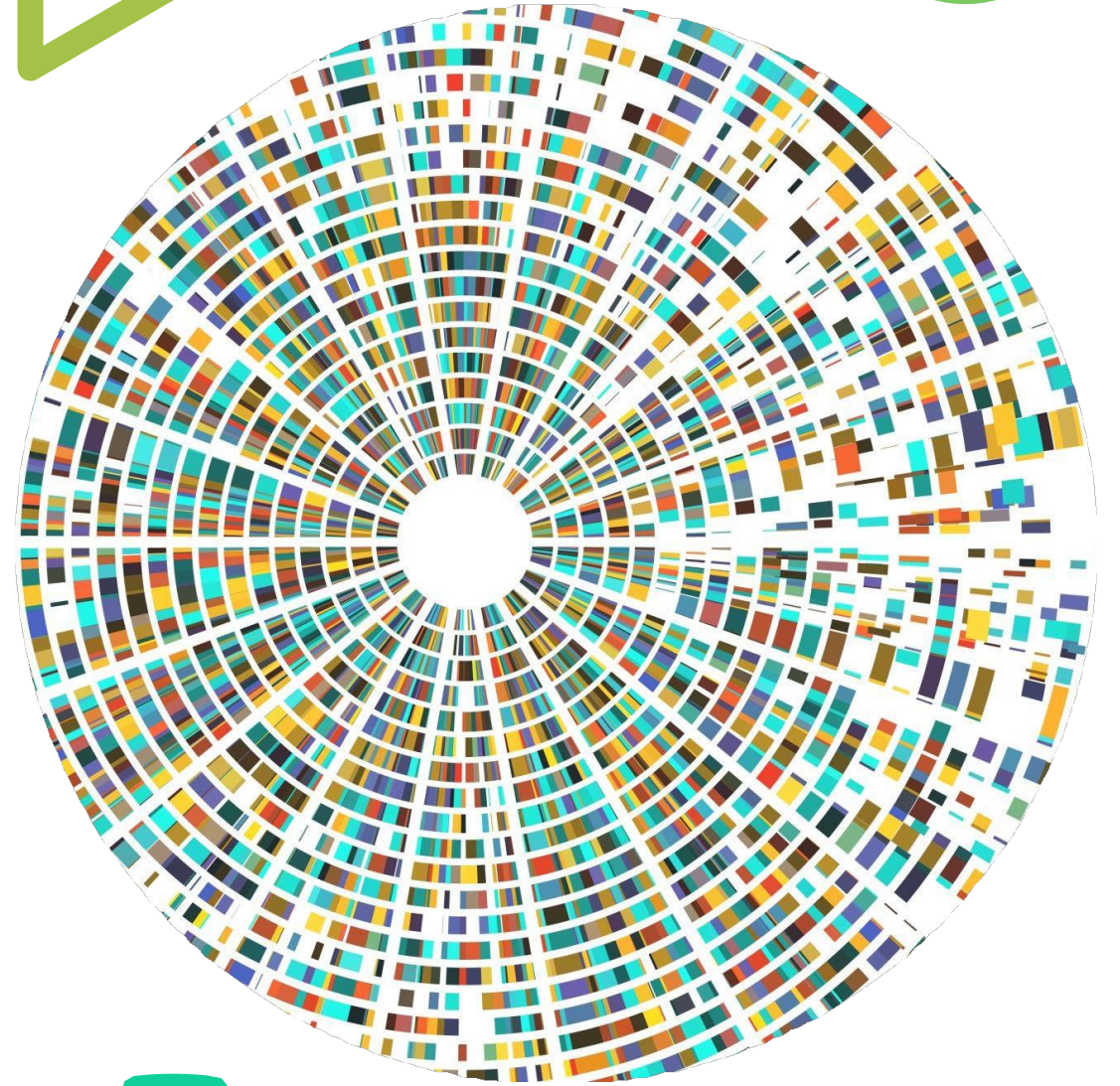




UT-5

Programación Orientada a Objetos

1º DAM/DAW
(2024-2025)



1 – Conceptos básicos POO

La Programación Orientada a Objetos (POO) hace que los problemas sean más sencillos al favorecer una relación fuerte entre datos y funciones. Esta relación se realiza a partir de estructuras denominadas objetos.

- Un **objeto** es un elemento del programa que posee sus propios datos y su propio funcionamiento.
- Una **clase** describe un grupo de objetos que contienen una información similar (atributos) y un comportamiento común (métodos).
- Antes de poder utilizar un objeto, se debe definir su clase. La clase es la definición de un tipo de objeto.

Este paradigma se basa en la idea de que cualquier programa está formado por objetos y que todo puede ser representado como tal. Así, cualquier elemento que forme parte de una aplicación (un usuario, una factura o pedido en un ERP, un coche para una aplicación de gestión de un taller, . . .) se considera que es un objeto de la aplicación con una serie de propiedades y características.

1 – Conceptos básicos P00

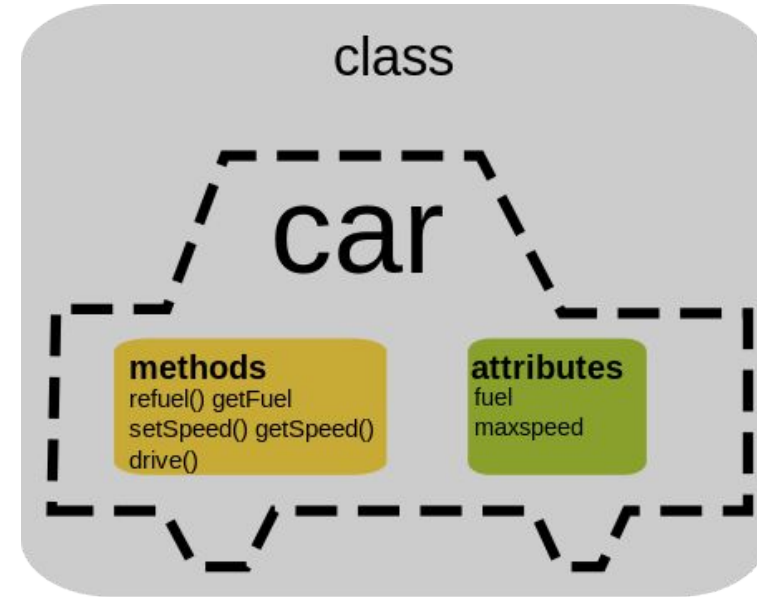
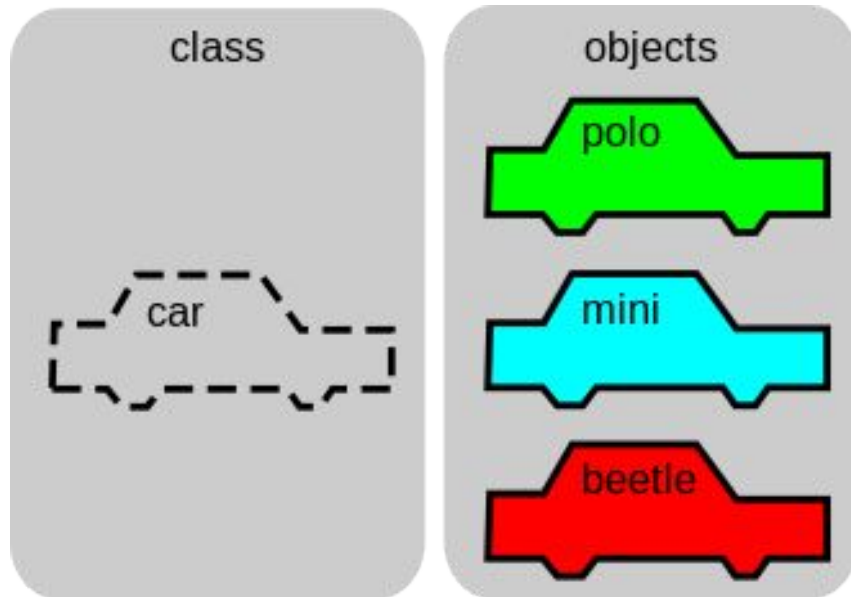
A partir del concepto de objeto, el paradigma propone una serie **características** que lo definen y que se irán comprendiendo a lo largo de este tema, según se vayan exponiendo los diferentes mecanismos de este paradigma para proporcionarlas:

- **Abstracción:** Por el que el programador se abstrae, se despreocupa, de los detalles de implementación de cualquier objeto. Los procesos o métodos que se encuentren definidos funcionan por sí solos y no es necesario saber cómo están implementados si sólo necesitamos hacer que se ejecuten.
- **Encapsulamiento:** Todas las características o propiedades que pertenezcan a un sólo elemento del programa se pueden crear y encapsular dentro de él, aumentando la cohesión de estos componentes.
- **Polimorfismo:** El polimorfismo nos permite utilizar a los objetos de manera genérica, aunque internamente se comporten según su variedad específica.
- **Herencia:** La herencia entre clases permitirá que una clase pueda heredar propiedades (atributos y métodos) de otra.

2 - Clases

Una clase describe un grupo de objetos que contienen una información similar (atributos) y un comportamiento común (métodos). Una clase es como un molde. A partir de ella se pueden crear objetos. Es decir, antes de poder utilizar un objeto se debe definir la clase a la que pertenece, esa definición incluye:

- **Atributos** . Las variables miembro de la clase.
- **Métodos** . Las funciones miembro de la clase.



2.1 – Clases (sintaxis)

```
[<modificador_visibilidad>] [<tipo>] class <NombreClase> {  
    [<atributos>]  
    [<constructores>]  
    [<getters/setters>]  
    [<métodos>]  
}
```

- Public.
- Private.
- Protected.

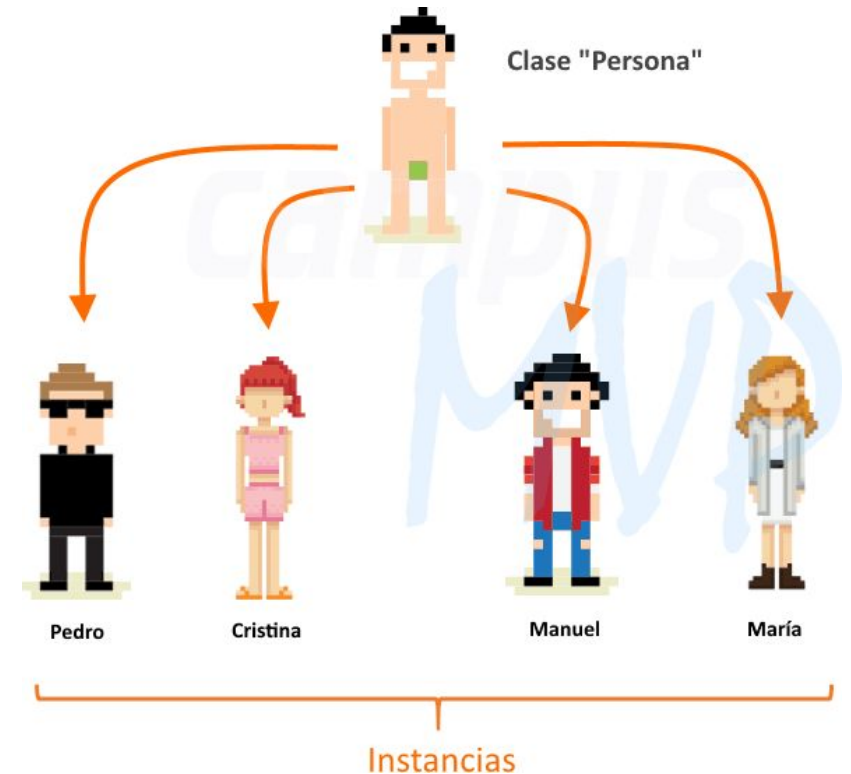
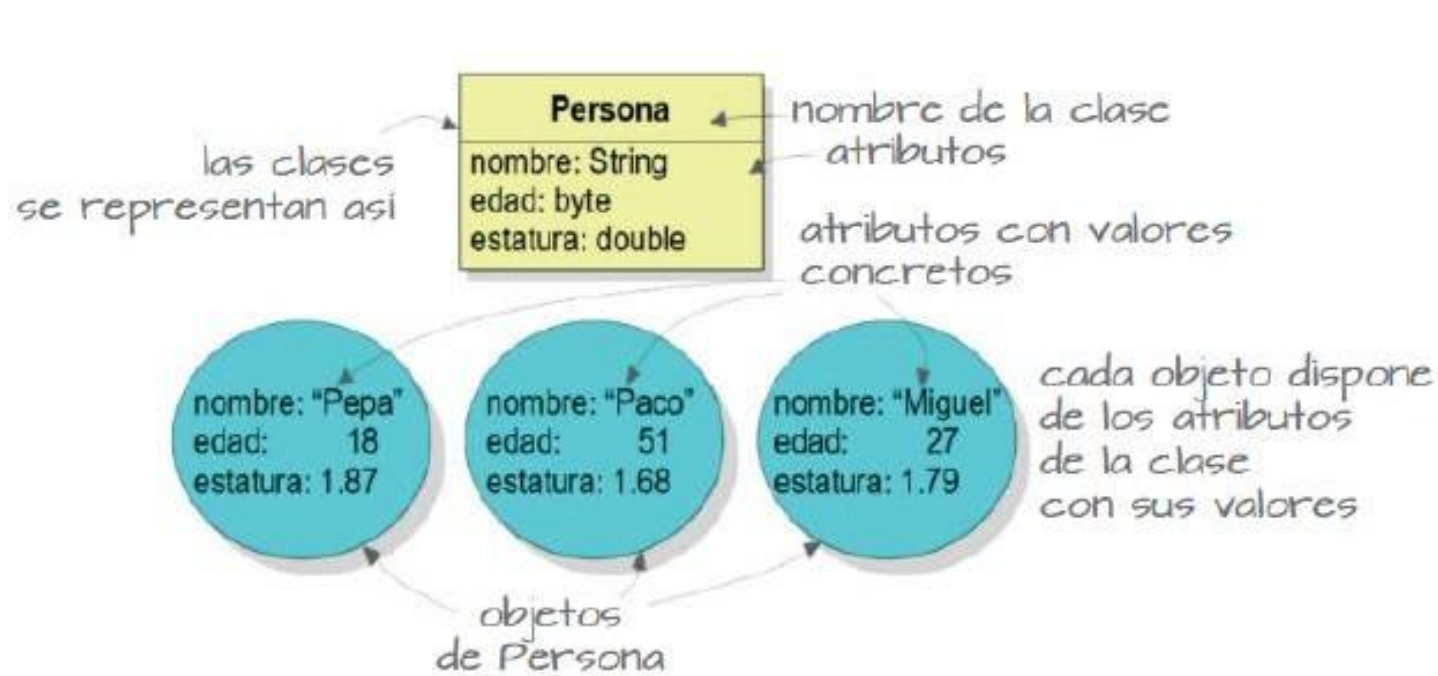
Notación UML

Nombre de la clase
Atributos
Métodos

```
public class Persona {  
  
    public String nombre;  
    public byte edad;  
    public double estatura;  
  
    public Persona () {  
        nombre = "Pepa";  
        edad = 18;  
        estatura = 1.87;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public void saludar() {  
        System.out.printf("\nHola me llamo %S y tengo %d años.\n", nombre, edad);  
    }  
}
```

3 - Objetos

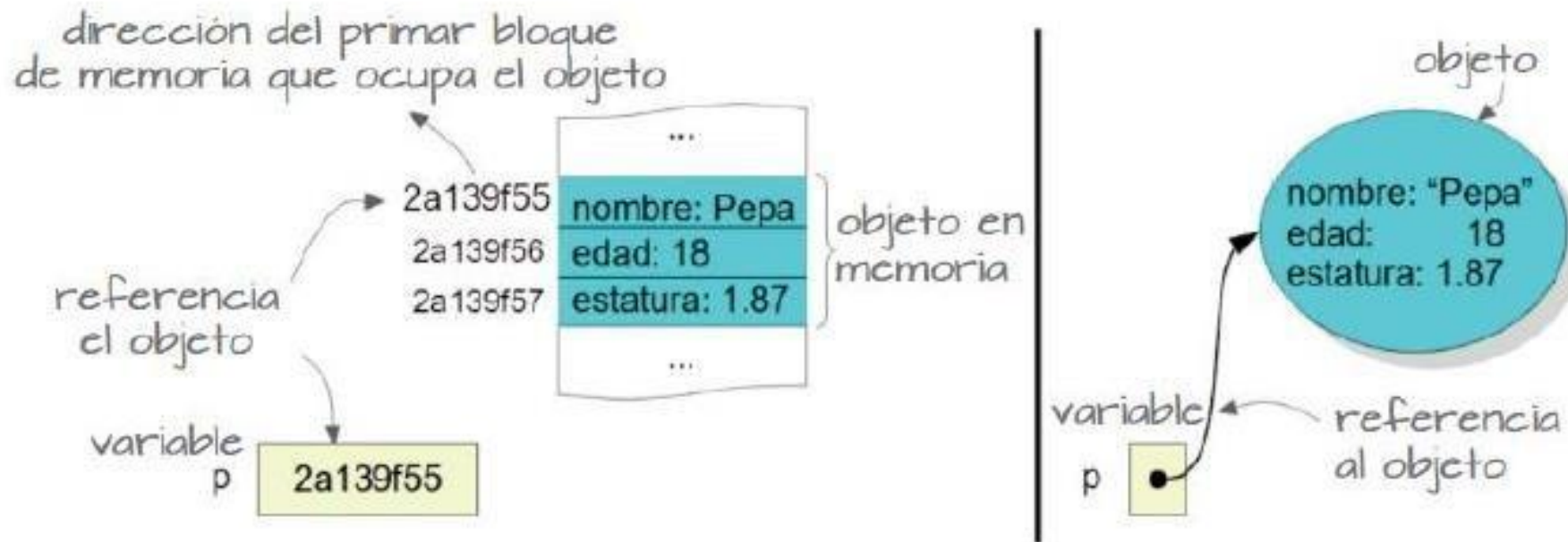
Los elementos pertenecientes a una clase se denominan instancias u objetos. Cada uno de ellos tendrá sus propios valores de los atributos definido en la clase.



3.1 – Objetos (Instanciación)

Antes de construir un objeto necesitamos declarar una variable cuyo tipo sea su clase. La forma de crear los objetos es mediante el operador **new**.

```
Persona p; // declaramos una variable de tipo Persona.  
p = new Persona(); // creamos el objeto y lo asignamos a la variable
```



3.2 – Objetos (referencias a null)

El valor literal **null** es una referencia nula. Cuando declaramos una variable de referencia se inicializa por defecto a null. Debemos tener la precaución de no acceder a los miembros de una referencia nula ya que esto lanzará una excepción “NullPointerException”.

```
Persona p; //se inicializa por defecto a null
p.nombre //¡error!
```

Los objetos no referenciados serán eliminados por el mecanismo automático de recolección de basura de Java. Un objeto se queda sin referencia cuando se asigna su variable de referencia a null o a otro objeto.

```
Persona p = new Persona("Pepa"); // La variable p referencia al objeto Pepa
p = new Persona("Juan"); // Ahora p referencia a otro objeto persona
// El objeto Pepa será eliminado por el recolector de basura.
```


4 – Visibilidad

Los miembros de una clase (atributos y métodos, es decir, sus variables y funciones) pueden definirse como **públicos, privados o protegidos**. Es importante entender la diferencia:

- **public**: Se puede utilizar desde cualquier clase.
- **private**: Sólo puede utilizarlo la propia clase.
- **protected**: Puede utilizarlo la propia clase y también las subclases heredadas.

Se llama **interfaz** a los miembros de una clase (atributos y métodos) que son public, porque son los que permiten interactuar con la clase desde fuera de ella. Los principios de la programación orientada a objetos indican que para mantener la encapsulación en los objetos debemos aplicar el especificador **public** a las funciones miembro que formen la interfaz pública y denegar el acceso a los datos miembro usados por esas funciones mediante el especificador **private**.

```
public class Persona {  
  
    private String nombre;  
    private byte edad;  
    private double estatura;  
  
    public void saludar() {  
        System.out.printf("\nHola me llamo %S y tengo %d años.\n", nombre, edad);  
        System.out.println("Encantado de conocerte.");  
    }  
    public void cumplirAños(){  
        this.edad++;  
    }  
    public void crecer(double incremento){  
        this.estatura+=incremento;  
    }  
}
```

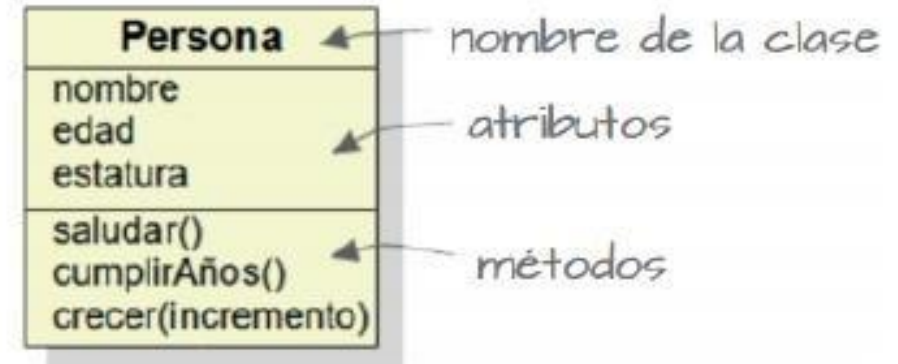
5 -Métodos

Las clases, además de atributos, también tienen un comportamiento que viene definido por sus métodos. Los métodos no son más que funciones que se implementan dentro de una clase.

```
public class Persona {  
  
    public String nombre;  
    public byte edad;  
    public double estatura;  
  
    public Persona () {  
        this.nombre = "Pepa";  
        edad = 18;  
        estatura = 1.87;  
    }  
}
```

```
    public void saludar() {  
        System.out.printf("\nHola me llamo %S y tengo %d años.\n",nombre,edad);  
        System.out.println("Encantado de conocerte.");  
    }  
    public void cumplirAños(){  
        this.edad++;  
    }  
    public void crecer(double incremento){  
        this.estatura+=incremento;  
    }  
}
```

Métodos



5.1 – Métodos (Constructores)

En programación orientada a objetos (POO), un **constructor es un método especial dentro de una clase que se utiliza para inicializar los objetos de esa clase**. En Java, los constructores tienen el mismo nombre que la clase y no tienen un tipo de retorno explícito.

El propósito principal de un constructor es asignar valores iniciales a los miembros de la clase o realizar cualquier otra inicialización necesaria cuando se crea un objeto. Los constructores son llamados automáticamente cuando se instancia un objeto de la clase. Este tipo de métodos permiten sobrecarga. Dependiendo de los parámetros que use el constructor, éste puede ser por defecto o parametrizado.

```
public class Persona {
```

```
    private String nombre;  
    private int edad;  
    private double estatura;
```

Constructores

```
    public Persona () {  
        this.nombre = "Pepa";  
        edad = 18;  
        estatura = 1.87;  
    }
```

```
    public Persona (String nombre) {  
        this();  
        this.nombre = nombre;  
    }
```

```
    public Persona (String nombre, int edad) {  
        this();  
        this.nombre = nombre;  
        this.edad=edad;  
    }
```

```
    public Persona (String nombre, int edad, double estatura) {  
        this.nombre = nombre;  
        this.edad=edad;  
        this.estatura=estatura;  
    }
```

5.2 – Métodos (Getter y Setter)

Puesto que lo más habitual es hacer que los atributos permanezcan lo más ocultos posible, se hace necesario de algún mecanismo que permita mostrarlos fuera de la implementación de la clase en el caso de que quieran ser leído o escritos desde nuestro proyecto. Para eso existen lo que se conoce como setters y getters. Los primeros permiten acceder a los atributos de una clase para modificarlos, mientras que los segundos permiten acceder a los mismos para leerlos.

Los getters y setters en la práctica no son más que métodos que siguen una notación determinada `getAtributo()` para los getters y `setAtributo(valor)` para los setters que permiten modificar o acceder a los atributos de una forma controlada.

Siempre será posible permitir que se pueda acceder a un atributo (getter) pero no para modificarlo.

```
public String getNombre() { return nombre; }

public void setNombre(String nombre) { this.nombre = nombre; }

public byte getEdad() { return edad; }

public void setEdad(byte edad) { this.edad = edad; }

public double getEstatura() { return estatura; }

public void setEstatura(double estatura) { this.estatura = estatura; }
```

6 – Atributos y Métodos estáticos

Los atributos y métodos estáticos que se definen en una clase como tal permiten que se puedan utilizar sin la necesidad de instanciar objetos de dicha clase. Se declaran con la palabra clave static.

Comparten datos comunes: Los atributos estáticos son útiles cuando deseamos que todas las instancias de una clase compartan un mismo valor.

Operaciones independientes: Los métodos estáticos son convenientes para operaciones que no dependen del estado específico de una instancia.

```
public class Persona {  
  
    private String nombre;  
    private int edad;  
    private double estatura;  
  
    // Atributo estático  
    private static int contadorPersonas = 0;  
  
    public Persona (String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad=edad;  
        contadorPersonas++; // Incrementar el contador cada vez que se crea una persona  
    }  
  
    // Método estático para obtener el número total de personas creadas  
    public static int obtenerContadorPersonas() {  
        return contadorPersonas;  
    }  
}
```


7 – Tipos Enumerados

Los tipos enumerados (enum) son una forma de definir un conjunto fijo de constantes con nombre. Los enums se introdujeron en Java para proporcionar una manera más robusta y legible de representar conjuntos de valores fijos. Puedes declarar un tipo enumerado mediante la palabra clave **enum**.

Los enums proporcionan una manera clara y segura de representar conjuntos de constantes relacionadas. Pueden mejorar la legibilidad y mantenimiento del código, y son especialmente útiles cuando necesitas representar un conjunto fijo de valores relacionados.

```
// Declaración de un tipo Enumerado
public enum DiaSemana {
    LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO
}
```

```
// Declaración de una variable de tipo enum
DiaSemana hoy = DiaSemana.MARTES;
```

```
// Comparación de enums
if (hoy == DiaSemana.MARTES) {
    System.out.println("Hoy es martes.");
}
```

```
// Iteración sobre todos los valores del enum
for (DiaSemana dia : DiaSemana.values()) {
    System.out.println(dia);
}
```

8 – Arrays de Objetos

En Java, los arrays de objetos permiten almacenar múltiples instancias de una clase en una única estructura de datos. Esto es especialmente útil cuando necesitas manejar conjuntos de objetos del mismo tipo de manera eficiente.

```
// Declaración e inicialización de un array de objetos
Persona [] grupo = new Persona[3];
// Creación de objetos y asignación al array
grupo[0]= new Persona("Juan");
grupo[1]= new Persona("Elisa");
grupo[2]= new Persona("Sara");
//Utilización de objetos
for (Persona p: grupo) {
    p.saludar();
}
```



```
Hola me llamo JUAN.
Encantado de conocerte.
```

```
Hola me llamo ELISA.
Encantado de conocerte.
```

```
Hola me llamo SARA.
Encantado de conocerte.
```